

Report (Q1 and Q2):

Q1. Minimum, Maximum, and Mean

Explanation of Approach:

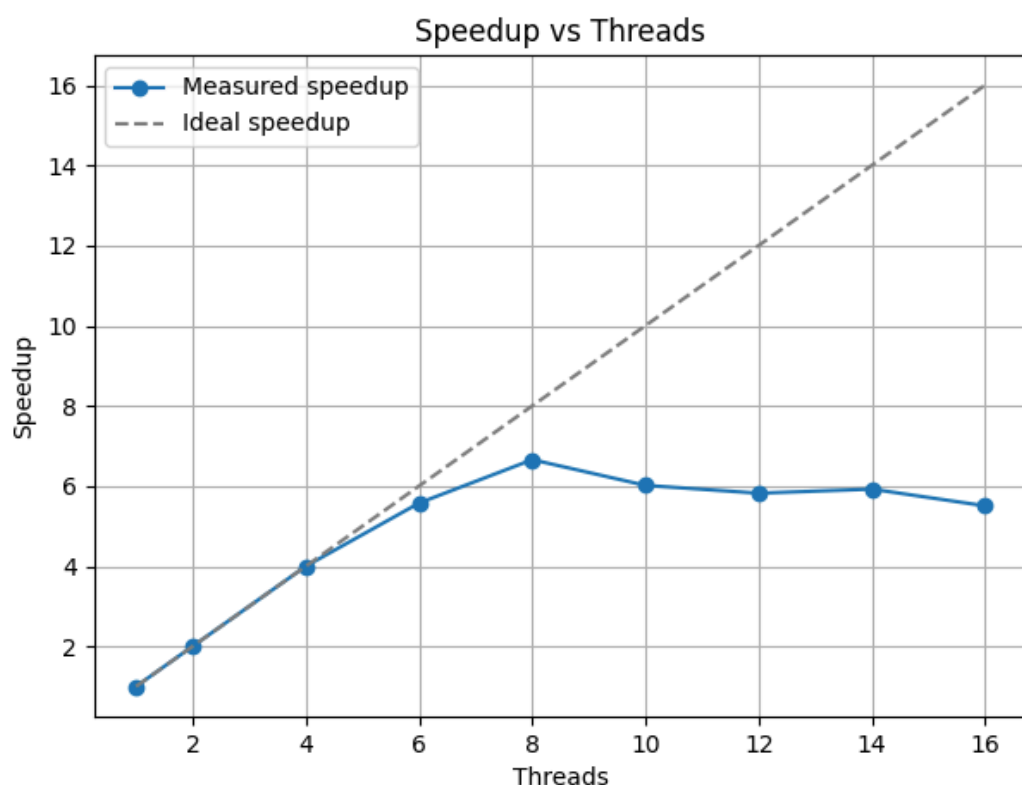
The C program computes the minimum, maximum, and mean values of a large sequence of randomly generated integers using OpenMP parallelization with chunking. Instead of storing the whole sequence, it generates numbers on the fly using a lightweight xorshift32 random number generator.

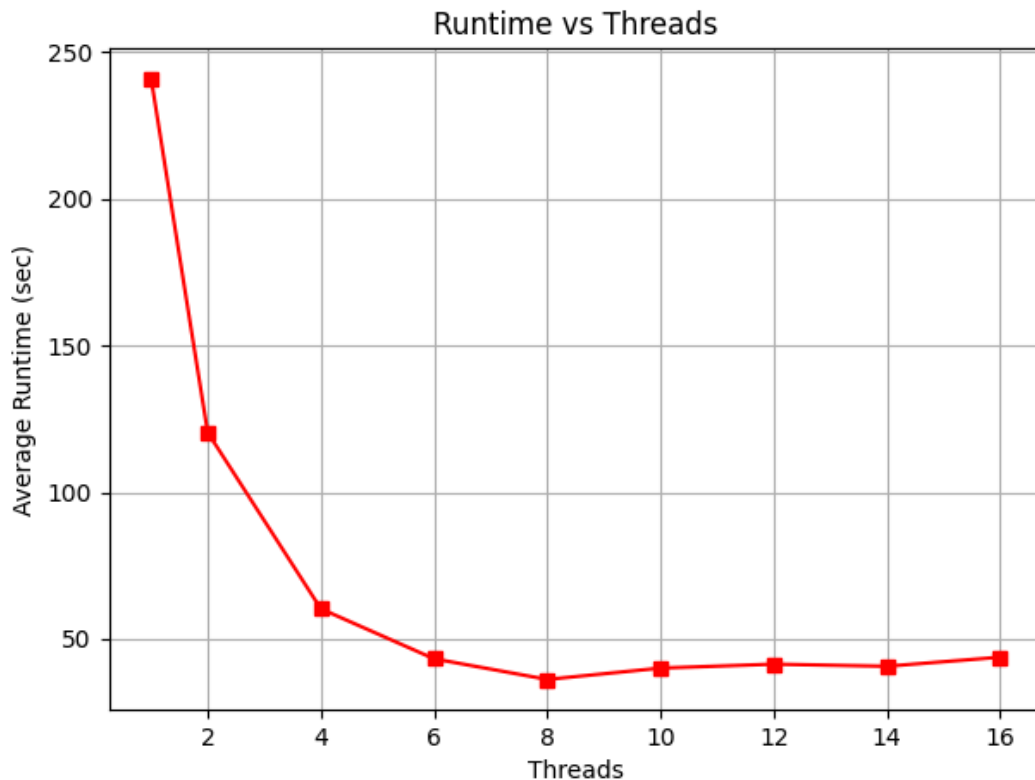
The computation is split into chunks (CHUNK_SIZE), and for each chunk, threads share the work in parallel. OpenMP reduction clauses are used to safely update the global_min, global_max, and global_sum across all threads. This way, the program avoids memory overhead while still processing billions of elements efficiently.

The program also benchmarks performance by running the calculation multiple times (RUNS) across different thread counts (thread_list). It measures execution time using gettimeofday, computes the average runtime, and calculates the speedup relative to the single-threaded baseline.

Results are printed in CSV format (threads, avg_time_sec, speedup), making it easy to analyze scaling behavior of the algorithm on multicore systems.

Result Obtained:





Q2. Dot Product

Explanation of Approach:

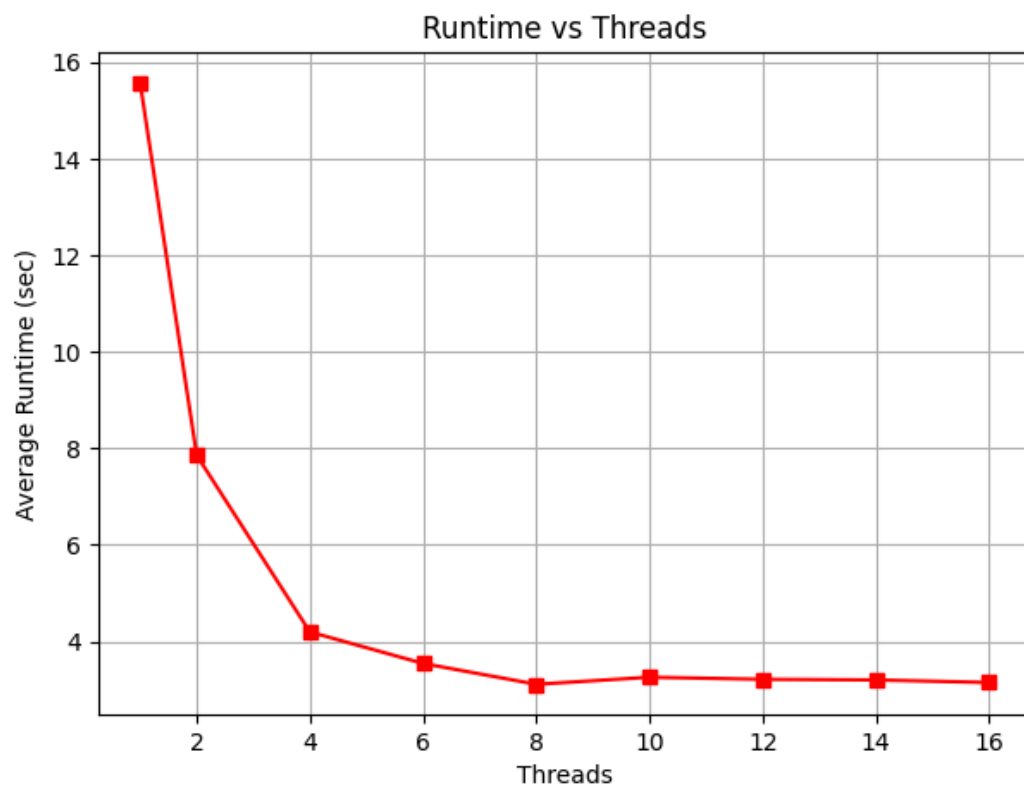
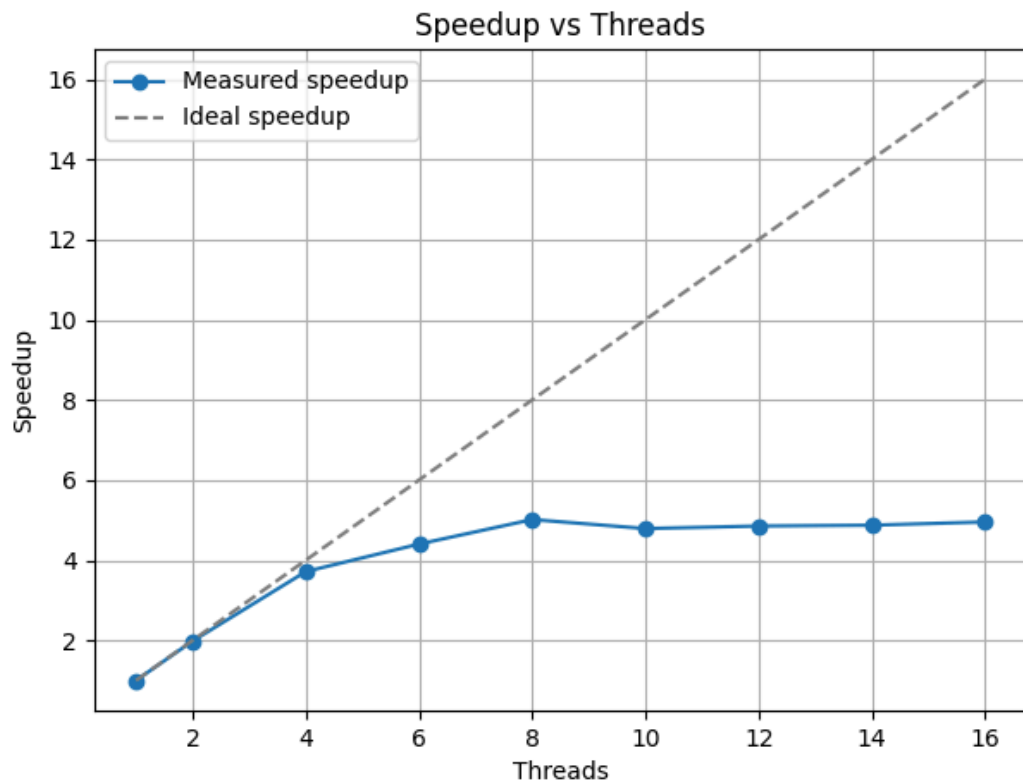
The program calculates the dot product of two very large vectors (size 109109) using OpenMP for parallelization and on-the-fly random generation. Instead of storing vectors, each element is generated dynamically using a fast xorshift random number generator.

The numbers are drawn from the set $\{-1, 0, 1\}$ via a custom function `rand_ternary`. The computation is chunked into blocks of 100 million elements to avoid memory overhead, and OpenMP's reduction clause is used so that multiple threads can safely update the shared `dot_product` result in parallel.

The program benchmarks performance across different thread counts (from 1 to 16), repeating the dot product calculation for multiple runs (`RUNS = 5`) to average out timing noise. It records the average runtime, calculates the speedup relative to the single-threaded case, and outputs results in CSV format: `threads, avg_time_sec, speedup, dot_product`.

This allows both verification of correctness (via the dot product value) and scalability analysis of the parallel algorithm.

Results Obtained:



Observations:

The graphs show the performance of your OpenMP parallel program on an 8-core Mac. From the runtime plot, execution time drops sharply as threads increase from 1 to 8, reaching the best runtime near 8 threads, which matches your hardware's core count. Beyond 8 threads, runtime either plateaus or slightly increases, since extra threads only add scheduling and synchronization overhead without real parallel benefit. This is also reflected in the speedup graphs: speedup grows nearly linearly up to 8 threads but flattens after that, never approaching the ideal linear scaling line.

In short, the program scales well up to the available cores, but adding more threads than physical cores provides no gain due to limited hardware parallelism.